SDN-Lab-1-Report

Lab Requirement

- 使用 `Mininet` 的Python API搭建 `k=4` 的 fat tree 拓扑；
- 使用 `pingall` 查看各主机之间的连通情况；
- 若主机之间未连通，分析原因并解决（使用 `wireshark` 抓包分析）
- 若主机连通，分析数据包的路径（提示：`ovs-appctl fdb/show`查看MAC表）
- 完成实验报告并提交到思源学堂
- 要求不能使用控制器

Environment

Arch Linux物理机

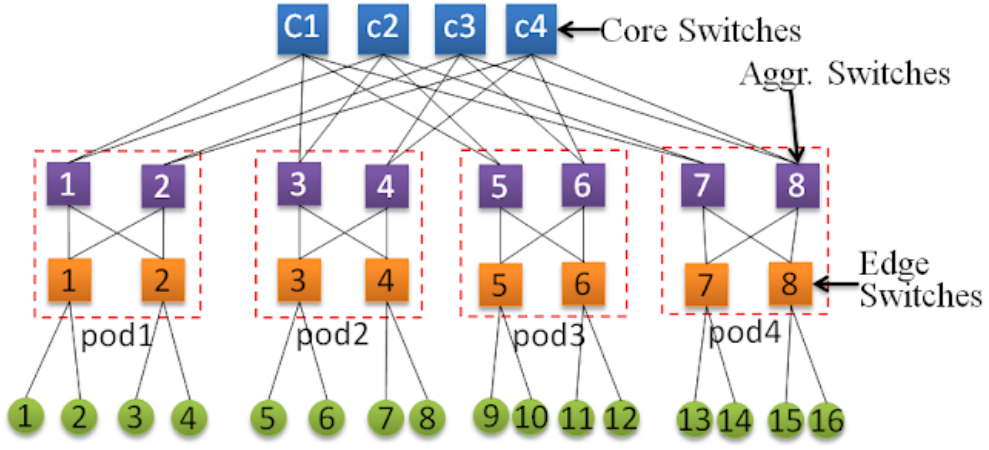
Topology Building

Topology Analysis

按照fat tree要求，每个交换机都有k个端口，核心交换机有 $\frac{k^2}{4}$ 个，下面有k个Pods，每个Pods里有$\frac{k}{2}$个汇聚交换机（Aggregation Switch），$\frac{k}{2}$个接入交换机，每个接入交换机（Edge Switch）下面连接$\frac{k}{2}$个主机。所以列出实际的结构：

- $\frac{k^2}{4}$，也就是4个核心交换机
- 4 个 pod
- 每个 pod 里有：
- 2 个接入交换机
- 2 个汇聚交换机
- 每个接入交换机下面连 2 台主机

  `k=4`的情况下，示意图如下：



所以按照这个方法，可以写出如下的代码。

```python
from mininet.topo import Topo
from mininet.cli import CLI
from mininet.node import OVSSwitch, Controller
from mininet.log import setLogLevel
from mininet.net import Mininet

class FatTreeTopo(Topo):
    def build(self, k=4):
        if k % 2 != 0:
            raise ValueError("输入偶数")
```

```
        pods = k
        half = k // 2

        core = []
        for i in range(half):
            row = []
            for j in range(half):
                switch = self.addSwitch(f'c{i}{j}') # 按列分组，每一列就是一个aggre交换机应该对应连接的core；共有half列，所以对应的就是\frac{k^2}{4}个交换机
                row.append(switch)
            core.append(row)

        for p in range(pods): # 创建每个pod里的edge和aggre交换机，并设置好连接
            aggre = []
            edg = []
            for a in range(half):
                aggre.append(self.addSwitch(f'a{p}{a}'))

            for e in range(half):
                edg.append(self.addSwitch(f'e{p}{e}'))

            for agg in aggre:
                for e in edg:
                    self.addLink(agg, e) # 添加边缘交换机和aggre交换机的连接，全连接

            for i, e in enumerate(edg):
                for h in range(half):
                    host = self.addHost(f'h{p}{i}{h}') # 第p个pod的第i个边缘交换机下的第h个主机
                    self.addLink(e, host)
            for j, agg in enumerate(aggre):
                for k in range(half):
                    self.addLink(agg, core[k][j]) # 第j个aggre交换机连接core交换机的第j列。可以看作core以列分组。

topos = {
    '4fattree': (lambda: FatTreeTopo(4))
}
def run():
    topo = FatTreeTopo(4)
    net = Mininet(topo)

    net.start()
    CLI(net)
    net.stop()


if __name__ == '__main__':
    setLogLevel('info') # info/output/debug
    run()
```

Test Connectivity

执行脚本

```bash
sudo python 4fattree.py
```

并用`pingall`测试连通性，输出如下：

```bash
*** Creating network
*** Adding controller
*** Adding hosts:
h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
```

```
*** Adding switches:
a00 a01 a10 a11 a20 a21 a30 a31 c00 c01 c10 c11 e00 e01 e10 e11 e20 e21 e30 e31
*** Adding links:
(a00, c00) (a00, c10) (a00, e00) (a00, e01) (a01, c01) (a01, c11) (a01, e00) (a01, e01) (a10, c00) (a10,
c10) (a10, e10) (a10, e11) (a11, c01) (a11, c11) (a11, e10) (a11, e11) (a20, c00) (a20, c10) (a20, e20)
(a20, e21) (a21, c01) (a21, c11) (a21, e20) (a21, e21) (a30, c00) (a30, c10) (a30, e30) (a30, e31) (a31,
c01) (a31, c11) (a31, e30) (a31, e31) (e00, h000) (e00, h001) (e01, h010) (e01, h011) (e10, h100) (e10,
h101) (e11, h110) (e11, h111) (e20, h200) (e20, h201) (e21, h210) (e21, h211) (e30, h300) (e30, h301)
(e31, h310) (e31, h311)
*** Configuring hosts
h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
*** Starting controller
c0
*** Starting 20 switches
a00 a01 a10 a11 a20 a21 a30 a31 c00 c01 c10 c11 e00 e01 e10 e11 e20 e21 e30 e31 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h000 -> h001 X X X X X X X X X X X X X X
h001 -> X X X X X X X X X X X X X X
h010 -> X X X X X X X X X X X X X X
h011 -> X X X X X X X X X X X X X X
h100 -> X X X X X X X X X X X X X X
h101 -> X X X X X X X X X X X X X X
h110 -> X X X X X X X X X X X X X X
h111 -> X X X X X X X X X X X X X X
h200 -> X X X X X X X X X X X X X X
h201 -> X X X X X X X X X X X X X X
h210 -> X X X X X X X X X X X X X X
h211 -> X X X X X X X X X X X X X X
h300 -> X X X X X X X X X X X X X X
h301 -> X X X X X X X X X X X X X X
h310 -> X X X X X X X X X X X X X X
h311 -> X X X X X X X X X X X X X X
*** Results: 99% dropped (1/240 received)
mininet>
```

主机之间无法连通。下面结合抓包结果来分析。

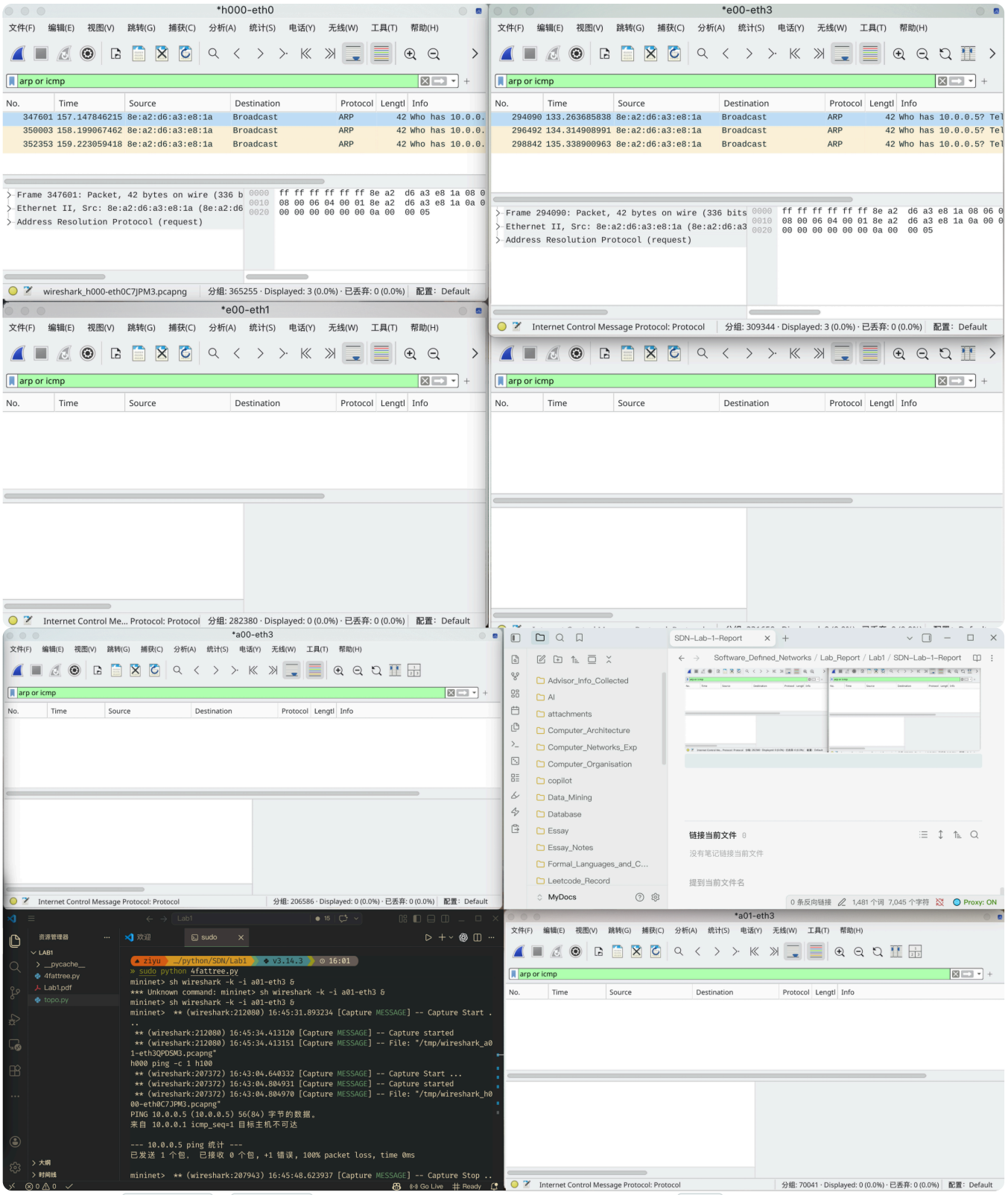
1. 首先查看实际网络拓扑

```bash
mininet> net
h000 h000-eth0:e00-eth3
h001 h001-eth0:e00-eth4
h010 h010-eth0:e01-eth3
h011 h011-eth0:e01-eth4
h100 h100-eth0:e10-eth3
h101 h101-eth0:e10-eth4
h110 h110-eth0:e11-eth3
h111 h111-eth0:e11-eth4
h200 h200-eth0:e20-eth3
h201 h201-eth0:e20-eth4
h210 h210-eth0:e21-eth3
h211 h211-eth0:e21-eth4
h300 h300-eth0:e30-eth3
h301 h301-eth0:e30-eth4
h310 h310-eth0:e31-eth3
h311 h311-eth0:e31-eth4
a00 lo:  a00-eth1:e00-eth1 a00-eth2:e01-eth1 a00-eth3:c00-eth1 a00-eth4:c10-eth1
a01 lo:  a01-eth1:e00-eth2 a01-eth2:e01-eth2 a01-eth3:c01-eth1 a01-eth4:c11-eth1
a10 lo:  a10-eth1:e10-eth1 a10-eth2:e11-eth1 a10-eth3:c00-eth2 a10-eth4:c10-eth2
a11 lo:  a11-eth1:e10-eth2 a11-eth2:e11-eth2 a11-eth3:c01-eth2 a11-eth4:c11-eth2
a20 lo:  a20-eth1:e20-eth1 a20-eth2:e21-eth1 a20-eth3:c00-eth3 a20-eth4:c10-eth3
a21 lo:  a21-eth1:e20-eth2 a21-eth2:e21-eth2 a21-eth3:c01-eth3 a21-eth4:c11-eth3
```

```
a30 lo:  a30-eth1:e30-eth1 a30-eth2:e31-eth1 a30-eth3:c00-eth4 a30-eth4:c10-eth4
a31 lo:  a31-eth1:e30-eth2 a31-eth2:e31-eth2 a31-eth3:c01-eth4 a31-eth4:c11-eth4
c00 lo:  c00-eth1:a00-eth3 c00-eth2:a10-eth3 c00-eth3:a20-eth3 c00-eth4:a30-eth3
c01 lo:  c01-eth1:a01-eth3 c01-eth2:a11-eth3 c01-eth3:a21-eth3 c01-eth4:a31-eth3
c10 lo:  c10-eth1:a00-eth4 c10-eth2:a10-eth4 c10-eth3:a20-eth4 c10-eth4:a30-eth4
c11 lo:  c11-eth1:a01-eth4 c11-eth2:a11-eth4 c11-eth3:a21-eth4 c11-eth4:a31-eth4
e00 lo:  e00-eth1:a00-eth1 e00-eth2:a01-eth1 e00-eth3:h000-eth0 e00-eth4:h001-eth0
e01 lo:  e01-eth1:a00-eth2 e01-eth2:a01-eth2 e01-eth3:h010-eth0 e01-eth4:h011-eth0
e10 lo:  e10-eth1:a10-eth1 e10-eth2:a11-eth1 e10-eth3:h100-eth0 e10-eth4:h101-eth0
e11 lo:  e11-eth1:a10-eth2 e11-eth2:a11-eth2 e11-eth3:h110-eth0 e11-eth4:h111-eth0
e20 lo:  e20-eth1:a20-eth1 e20-eth2:a21-eth1 e20-eth3:h200-eth0 e20-eth4:h201-eth0
e21 lo:  e21-eth1:a20-eth2 e21-eth2:a21-eth2 e21-eth3:h210-eth0 e21-eth4:h211-eth0
e30 lo:  e30-eth1:a30-eth1 e30-eth2:a31-eth1 e30-eth3:h300-eth0 e30-eth4:h301-eth0
e31 lo:  e31-eth1:a30-eth2 e31-eth2:a31-eth2 e31-eth3:h310-eth0 e31-eth4:h311-eth0
c0
```

2. 所以可以看到，如果要让`h000`去`ping` `h001`，应该监视的位置为：`h000→eth0` `e00-eth3` `e00→eth1` `e00→eth2` `a00→eth3` `a01→eth3`。因此执行下列命令，并保存抓包结果

```bash
mininet> h000 wireshark -k -i h000-eth0 &
mininet> sh wireshark -k -i e00-eth3 &
mininet> sh wireshark -k -i e00-eth1 &
mininet> sh wireshark -k -i e00-eth2 &
mininet> sh wireshark -k -i a00-eth3 &
mininet> sh wireshark -k -i a01-eth3 &
mininet> h000 ping -c 1 h100
```



结果显示，只有`h000-eth0`和`e00-eth3`的抓包结果里有ARP协议包。这可能说明源主机 `h000` 发出的 ARP 请求能够到达接入交换机 `e00`，但 `e00` 未将该 ARP 帧从上行端口继续转发，因此 ARP 无法到达目标主机所在网段，后续 ICMP 无法进行。

而这个结果也是可以预料的，因为目前的代码用的是默认的`Mininet(topo)`，所以应该采用了默认的`OVSSwitch`和默认的控制器，**因此和实验要求不符**。下面检查OVS的各种命令

```bash
mininet> sh ovs-vsctl get bridge e00 fail_mode
secure
mininet> sh ovs-vsctl get-controller e00
tcp:127.0.0.1:6653
mininet> sh ovs-ofctl dump-flows e00
```

可以看到，OVS的工作状态是`secure`，也就是等待控制器下发转发规则，而不会自己学习。所以当前网络仍依赖默认控制器/默认 OVS 转发模式，未满足实验要求中的无控制器条件，因此主机间无法连通。
所以下面对代码做出修改(主要是`run()`)。(也可以通过文档中参考的做法在CLI中修改OVS的属性)

Implementation

```Python
from functools import partial

from mininet.topo import Topo
from mininet.cli import CLI
from mininet.node import OVSBridge
from mininet.log import setLogLevel
from mininet.net import Mininet

class FatTreeTopo(Topo):
    def build(self, k=4):
        if k % 2 != 0:
            raise ValueError("k must be even")

        pods = k
        half = k // 2

        core = []
        for i in range(half):
            row = []
            for j in range(half):
                switch = self.addSwitch(f'c{i}{j}')
                row.append(switch)
            core.append(row)

        for p in range(pods):
            aggre = []
            edg = []
            for a in range(half):
                aggre.append(self.addSwitch(f'a{p}{a}'))

            for e in range(half):
                edg.append(self.addSwitch(f'e{p}{e}'))

            for agg in aggre:
                for e in edg:
                    self.addLink(agg, e)

            for i, e in enumerate(edg):
                for h in range(half):
                    host = self.addHost(f'h{p}{i}{h}')
                    self.addLink(e, host)
            for j, agg in enumerate(aggre):
                for c_idx in range(half):
                    self.addLink(agg, core[c_idx][j])


topos = {
    '4fattree': lambda: FatTreeTopo(4)
}


def run():
    topo = FatTreeTopo(4)
    net = Mininet(
        topo=topo,
        switch=partial(OVSBridge, stp=True), # 设置生成树协议
        controller=None, # 取消控制器
```

```
        autoSetMacs=True, # 给主机自动分配MAC，optional
        waitConnected=True,
    )

    net.start()
    CLI(net)
    net.stop()


if __name__ == '__main__':
    setLogLevel('info')
    run()
```

Result

这下检查控制器和OVS状态，发现符合我们的期望：

```bash
» sudo python 4fattree.py
*** Creating network
*** Adding hosts:
h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
*** Adding switches:
a00 a01 a10 a11 a20 a21 a30 a31 c00 c01 c10 c11 e00 e01 e10 e11 e20 e21 e30 e31
*** Adding links:
(a00, c00) (a00, c10) (a00, e00) (a00, e01) (a01, c01) (a01, c11) (a01, e00) (a01, e01) (a10, c00) (a10,
c10) (a10, e10) (a10, e11) (a11, c01) (a11, c11) (a11, e10) (a11, e11) (a20, c00) (a20, c10) (a20, e20)
(a20, e21) (a21, c01) (a21, c11) (a21, e20) (a21, e21) (a30, c00) (a30, c10) (a30, e30) (a30, e31) (a31,
c01) (a31, c11) (a31, e30) (a31, e31) (e00, h000) (e00, h001) (e01, h010) (e01, h011) (e10, h100) (e10,
h101) (e11, h110) (e11, h111) (e20, h200) (e20, h201) (e21, h210) (e21, h211) (e30, h300) (e30, h301)
(e31, h310) (e31, h311)
*** Configuring hosts
h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
*** Starting controller

*** Starting 20 switches
a00 a01 a10 a11 a20 a21 a30 a31 c00 c01 c10 c11 e00 e01 e10 e11 e20 e21 e30 e31 ...
*** Waiting for switches to connect
a00 a01 a10 a11 a20 a21 a31 c01 c11 e01 e10 e11 e20 e30 e31 a30 e00 c00 c10 e21
*** Starting CLI:
mininet> sh ovs-vsctl get-controller e00
mininet> sh ovs-vsctl get bridge e00 stp_enable
true
mininet> sh ovs-vsctl get-fail-mode e00
standalone
```

下面开始测试。

```bash
mininet> pingall
*** Ping: testing ping reachability
h000 -> h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h001 -> h000 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h010 -> h000 h001 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h011 -> h000 h001 h010 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h100 -> h000 h001 h010 h011 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h101 -> h000 h001 h010 h011 h100 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h110 -> h000 h001 h010 h011 h100 h101 h111 h200 h201 h210 h211 h300 h301 h310 h311
h111 -> h000 h001 h010 h011 h100 h101 h110 h200 h201 h210 h211 h300 h301 h310 h311
h200 -> h000 h001 h010 h011 h100 h101 h110 h111 h201 h210 h211 h300 h301 h310 h311
h201 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h210 h211 h300 h301 h310 h311
h210 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h211 h300 h301 h310 h311
h211 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h300 h301 h310 h311
```

```
h300 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h301 h310 h311
h301 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h310 h311
h310 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h311
h311 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310
*** Results: 0% dropped (240/240 received)
```

这次成功了。

查看各个设备的MAC地址

```bash
mininet> py [print(h.name, h.MAC()) for h in net.hosts]
h000 00:00:00:00:00:01
h001 00:00:00:00:00:02
h010 00:00:00:00:00:03
h011 00:00:00:00:00:04
h100 00:00:00:00:00:05
h101 00:00:00:00:00:06
h110 00:00:00:00:00:07
h111 00:00:00:00:00:08
h200 00:00:00:00:00:09
h201 00:00:00:00:00:0a
h210 00:00:00:00:00:0b
h211 00:00:00:00:00:0c
h300 00:00:00:00:00:0d
h301 00:00:00:00:00:0e
h310 00:00:00:00:00:0f
h311 00:00:00:00:00:10
[None, None, None, None, None, None, None, None, None, None, None, None, None, None, None, None]
```

Transmission Analysis

下面以`h000`到`h100`的`ping`指令为例分析一下此情境下数据包的路径。

```bash
mininet> sh ovs-appctl fdb/show c01
 port  VLAN  MAC                Age
mininet> h000 ping -c 1 h100
PING 10.0.0.5 (10.0.0.5) 56(84) 字节的数据。
64 字节，来自 10.0.0.5: icmp_seq=1 ttl=64 时间=2.27 毫秒

--- 10.0.0.5 ping 统计 ---
已发送 1 个包， 已接收 1 个包, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 2.265/2.265/2.265/0.000 ms
mininet> sh ovs-appctl fdb/show e00
 port  VLAN  MAC                Age
    3     0  00:00:00:00:00:01    8
    2     0  00:00:00:00:00:05    8
mininet> sh ovs-appctl fdb/show e01
 port  VLAN  MAC                Age
    2     0  00:00:00:00:00:01   17
mininet> sh ovs-appctl fdb/show a00
 port  VLAN  MAC                Age
    2     0  00:00:00:00:00:01   22
mininet> sh ovs-appctl fdb/show a01
 port  VLAN  MAC                Age
    1     0  00:00:00:00:00:01   17
    4     0  00:00:00:00:00:05   17
mininet> sh ovs-appctl fdb/show c00
 port  VLAN  MAC                Age
    4     0  00:00:00:00:00:01   28
mininet> sh ovs-appctl fdb/show c01
 port  VLAN  MAC                Age
    4     0  00:00:00:00:00:01   30
mininet> sh ovs-appctl fdb/show c11
```

```
 port  VLAN  MAC                Age
    1     0  00:00:00:00:00:01   59
    2     0  00:00:00:00:00:05   59
mininet> sh ovs-appctl fdb/show a11
 port  VLAN  MAC                Age
    1     0  00:00:00:00:00:05   68
    4     0  00:00:00:00:00:01   68
mininet> sh ovs-appctl fdb/show e10
 port  VLAN  MAC                Age
    3     0  00:00:00:00:00:05  260
    2     0  00:00:00:00:00:01  260
mininet> sh ovs-appctl fdb/show e11
 port  VLAN  MAC                Age
    2     0  00:00:00:00:00:01  268
```

可以看到，实际上的转发路径为：

`h000 → e00 → a01 → c11 → a11 → e10 → h100`

做出这个判断的依据是，`h000`请求通信的时候是先发送ARP Request广播帧，然后帧沿着生成树洪泛传播，各个交换机因此都能学到它的MAC地址；而真正的`h100`设备以单播的形式发回ARP Reply，随后的ICMP数据包都是通过单播的形式进行。所以观察到哪个交换机里有它的MAC，就说明它是这个数据包路径中的一个node。这也可以通过wireshark抓包得到进一步佐证，不过证据已经足够强。